ByteDance | Seed

Efficient Pretraining Length Scaling

Bohong Wu[1,†], **Shen Yan**[1,2], **Sijun Zhang**[1], **Jianqiao Lu**[1,3], **Yutao Zeng**[1], **Ya Wang**[1], **Xun Zhou**[1]

[1]ByteDance Seed, [2]Peking University, [3]Hong Kong University

[*]Work done at ByteDance Seed, [†]Corresponding authors

Abstract

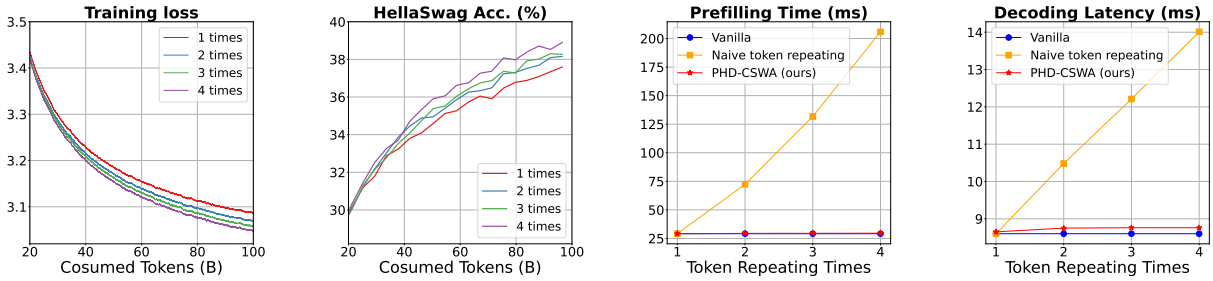
Recent advances in large language models have demonstrated the effectiveness of length scaling during post-training, yet its potential in pre-training remains underexplored. We present the Parallel Hidden Decoding Transformer (*PHD*-Transformer), a novel framework that enables efficient length scaling during pre-training while maintaining inference efficiency. *PHD*-Transformer achieves this through an innovative KV cache management strategy that distinguishes between original tokens and hidden decoding tokens. By retaining only the KV cache of original tokens for long-range dependencies while immediately discarding hidden decoding tokens after use, our approach maintains the same KV cache size as the vanilla transformer while enabling effective length scaling. To further enhance performance, we introduce two optimized variants: *PHD-SWA* employs sliding window attention to preserve local dependencies, while *PHD-CSWA* implements chunk-wise sliding window attention to eliminate linear growth in pre-filling time. Extensive experiments demonstrate consistent improvements across multiple benchmarks.

Date: April 22, 2025
Correspondence: Bohong Wu at bohongwu@bytedance.com, Xun Zhou at zhouxun@bytedance.com

1 Introduction

Recent years have witnessed the breakthrough of large language models (LLMs) [24, 31, 33, 34, 51, 52] across various domains [6, 19, 27, 54]. Apart from the early success of scaling in parameters and training data [9, 16, 22, 28] in the pre-training stage, the recent success of Deepseek-R1 [18] and OpenAI-o1/o3 [38, 39] have stimulated researches on length scaling during the post-training stage. By employing RL methods including PPO [44] or GRPO [46], model learns length scaling by generating very long chain-of-thought (COT) [55] sequences before giving the final answers, leading to remarkable improvement on Olympiad-level math and reasoning problems including American Invitational Mathematics Examination (AIME) and GPQA [41].

Given the great success of length scaling in post-training, researchers have also been studying length scaling in the pre-training stage. Early work on chain-of-thought reasoning [55] inspires methods that insert plain text into pre-training sequences either through manual design [17] or online exploration [59]. Recently, under the concept of converting discrete signals into continuous representations, Coconut [20] proposes inserting latent embeddings rather than plain text. CoTFormer [36] achieves an implicit 2× pre-training length scaling by reusing hidden embeddings from earlier layers for a single token. In contrast, COCOMix [48] emphasizes the interpretability of intermediate hidden states and projects them into continuous concepts.



(a) Training loss. **(b)** HellaSwag accuracy. **(c)** Prefilling time. **(d)** Decoding latency.

Figure 1 The length scaling curve on a 151M sized model. We repeat the training sequence 1/2/3/4 times on the same model architecture and train them for 100B tokens. The training loss and downstream accuracy scale robustly w.r.t. the token repeating times.

Although these pioneer researches are proven effective, they have limited applicability, as their marginal improvements in reasoning benchmarks are achieved at the cost of increased KV cache size and higher decoding latency. Moreover, distracted by post-processing the middle layer hidden states in various ways, the innate pre-training length scaling phenomenon is never explored.

More essentially, we present that length scaling can also be achieved in pre-training. Unlike previous works [36, 48], we simply repeat the input tokens 1/2/3/4 times without post-processing on middle layer hidden states. We observe both the loss scaling and performance scaling trend w.r.t. the token repeating times, which is shown in Figure 1a and Figure 1b. However, naively repeating input tokens incurs significant inefficiencies in inference. The key obstacle arises from the linearly increased KV cache size due to token repetition, which introduces both memory pressure from the KV cache footprint, super linear increase in pre-filling time (shown in Figure 1c), and linear increase in decoding latency (shown in Figure 1d).

To address these challenges, We present a novel inference-friendly length scaling approach. Our key contribution is the Parallel Hidden Decoding Transformer (*PHD*-Transformer), which maintains the same KV cache size as the vanilla transformer while enabling effective length scaling. The *PHD*-Transformer achieves this through an innovative KV cache management strategy. Specifically, denote the first tokens as *original tokens*, and the repeated tokens as *hidden decoding tokens*, we exclusively retain the KV cache generated from *original tokens* for long-range dependency modeling and immediately discard the KV cache of *hidden decoding tokens* after their use in next-token prediction. This approach provides an identical KV cache size to the vanilla Transformer while delivering substantial inference acceleration compared to naive token repeating (shown in Figure 1d).

To better preserve the performance benefits from the KV cache of *hidden decoding tokens*, we introduce *PHD-SWA* (Sliding Window Attention). This variant maintains a local sliding window cache of these tokens, achieving notable performance improvements while requiring only $\mathcal{O}(1)$ additional KV cache memory. However, we notice that the KV cache of *hidden decoding tokens* exhibits sequential dependencies in *PHD-SWA*, which leads to a linear increase in the pre-filling time. To address this, we propose *PHD-CSWA* (Chunk-wise Sliding Window Attention), which restricts the sequential dependencies within each chunk. Therefore, *PHD-CSWA* significantly reduces the pre-filling time as only the pre-filling time of the last chunk is linearly increased (shown in Figure 1c).

In summary, we propose our parallel hidden decoding-transformer series, including *PHD*, *PHD-SWA*, and *PHD-CSWA*. To our knowledge, this work first presents the topic of efficient pre-training length scaling. Our contributions are summarized as follows:

- We propose our *PHD*-Transformer series, which not only demonstrates the effectiveness of length scaling but also provides novel solutions to prevent the linear growth of KV cache size.
- Through comprehensive empirical evaluation, we demonstrate that our PHD-Transformer series delivers substantial accuracy improvements across multiple benchmarks while maintaining acceptable

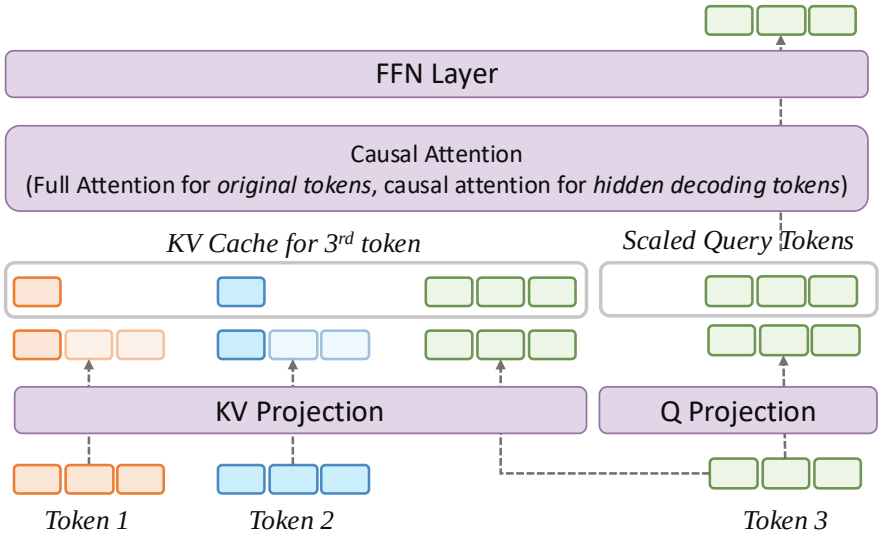


Figure 2 Overview of the transformer block in *PHD*. Specifically, the input tokens are repeated multiple times fed into the transformer block simultaneously. The *original tokens* generate KV cache that can be attended to by all the following tokens, while the *hidden decoding tokens* only generate KV cache that can be attended to within the current tokens (*Token 3* in the Figure).

computational overhead in both the pre-filling and decoding phases.

2 Approach

In this section, we propose our *PHD*-Transformer series, including *PHD*, its sliding window attention variant *PHD-SWA*, and its chunk-wise sliding window attention variant *PHD-CSWA*.

2.1 Notations

We set up the notations in this section. Assume a training sample $x \in X$ contains t tokens $\{x_1, x_2, ..., x_t\}$, where the hidden representation of each token can be represented as $\mathbf{h} = \{h_1, h_2, ..., h_t\}$, and the hidden dimension of each token is d. Let M_{mn} be the attention mask between x_m and x_n, the original self-attention within sample x can be written as follows:

$$\text{Self-Attention}(\mathbf{h}) = \text{softmax}\left(\frac{(W_q\mathbf{h})(W_k\mathbf{h})^\top \otimes M}{\sqrt{d}}\right)(W_v\mathbf{h}). \tag{1}$$

In our proposed *PHD*, we repeat tokens K times, where we extend the sequence to

$$\{x_1^1, x_1^2, ...x_1^K;\quad x_2^1, x_2^2, ..., x_2^K;\quad ...;\quad x_t^1, x_t^2, ..., x_t^K\}.$$

For better illustration, we name $x_j^1, 1 \le j \le t$ as the *origin tokens*, and $x_j^s, 1 \le j \le t, 2 \le s \le K$ as the *hidden decoding tokens*. We denote M_{mn}^{ij} as the attention mask between x_m^i and x_n^j.

2.2 PHD

The architecture of *PHD* is mainly presented in Figure 2. Compared to the original transformer, *PHD* keeps the same model architecture and only differs in the input sequence and the design of the attention matrix. Specifically, We only allow the *origin tokens* $x_j^1, 1 \le j \le t$ to generate the KV cache and can be globally attended to by all tokens, while the KV cache of hidden states is dropped instantly after parallel hidden decoding. The attention matrix strategy is formulated as follows:

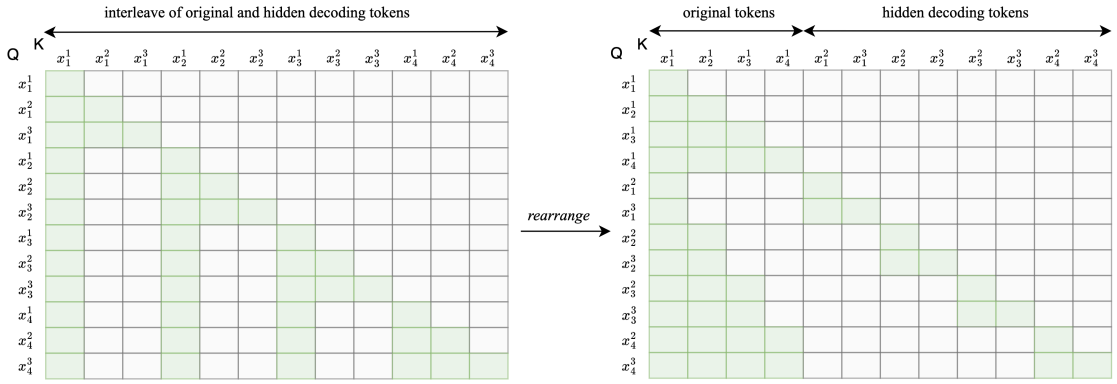


Figure 3 The attention matrix in *PHD*. The interleaving of *original tokens* and *hidden decoding tokens* introduce very sparse attention matrix that is not device friendly. We rearrange the input sequence and split the original tokens and hidden decoding tokens into two groups. In this way, we group the un-attended attention positions in a continuous block, which is efficient for optimization.

$$M_{mn}^{ij} = \begin{cases} 1, & \textit{if } i = 1 \textit{ and } m < n \\ 1, & \textit{if } i \leq j \textit{ and } m = n \\ 0. & \textit{otherwise} \end{cases} \tag{2}$$

Our design achieves identical KV cache size and memory access patterns as the original Transformer during inference. While requiring K times FLOPs, these computations can be processed in parallel, resulting in minimal latency overhead in memory-bound inference scenarios. The key advantage of architecture stems from the decoupling between *original tokens* and *hidden decoding tokens*. During pre-filling, only *original tokens* require computation. This design ensures pre-filling time is the same as the vanilla transformer and remains constant regardless of the scaling factor K.

2.3 Kernel Design

Naive implementation of M_{mn}^{ij} results into K^2 times computation increase in the attention layers and K times increase in the FFN layers. However, since the attention is sparsely computed, the $\mathcal{O}(K^2)$ attention can be largely reduced. Consequently, we split the *original tokens* and *hidden decoding tokens* into two groups, and concatenate them together. Figure 3 shows an example of $K = 3$, where we can get one sequence of t original tokens and one sequence of $2t$ hidden decoding sequence. By rearranging the token positions, we keep the attention positions that are masked in one continuous block, which leads to optimization in attention computation, reducing the complexity of attention computation to $\mathcal{O}(K)$.

2.4 PHD-SWA and PHD-CSWA

Compared to naive token repeating, our *PHD*-Transformer achieves length scaling while maintaining the original KV cache size. However, we empirically observe that preserving some KV cache for *hidden decoding tokens* yields significant performance benefits. To capture these benefits while maintaining efficiency, we introduce PHD-SWA, which implements sliding window attention restricted to W preceding *hidden decoding tokens*. As illustrated in Figure 4, the attention pattern combines global access to original tokens with local access to the W most recent *hidden decoding tokens*. This modified attention mechanism achieves notable performance improvements while only requiring $\mathcal{O}(1)$ additional KV cache memory.

While the sliding window approach in *PHD-SWA* enhances model performance, it incurs a K times pre-filling overhead caused by sequential dependencies in the KV cache of *hidden decoding tokens*. To address this, we introduce *PHD-CSWA* which processes attention within independent chunks. As demonstrated in Figure 4, *PHD-CSWA* constrains the sliding window attention to operate within individual chunks. This architectural

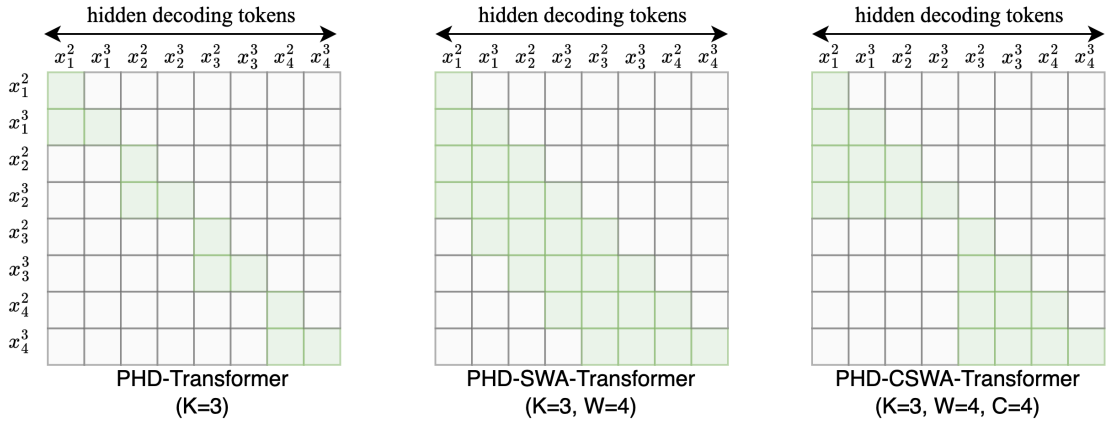


Figure 4 Comparison of the attention matrices in *PHD*, *PHD-SWA* and *PHD-CSWA*. In this figure, we set the repeating times K to 3, which means there are 2 hidden decoding tokens in each attention matrix, and set the window size W to 4 and chunk size C to 4.

innovation reduces the extra pre-filling overhead to just K repetitions within the final chunk, rather than across the entire sequence, making the additional computational cost practically negligible while preserving the benefits of local attention patterns.

3 Experiments

In this section, we present a detailed experimental analysis of our proposed *PHD*. We use OLMo2 [37] as the codebase of all our experiments. The hyperparameters and details are illustrated in each subsection correspondingly. All model variants are named in the format of *model-type-K-W-C* format for better illustration.

3.1 Evaluation Metrics

To evaluate the performance of our proposed *PHD-Transformer* series, we evaluate them on the following set of open benchmarks, including ARC [10], HellaSwag [60], PIQA [4], Winogrande [43], MMLU [21], and CommonsenseQA [49].

3.2 Main Results

In this section, we present the main results to show the efficacy of our *PHD-CSWA* as it is the most practical and effective strategy, which introduces steady performance improvement with acceptable overhead.

Training Details We use the 1.2B-sized model, which is a 16-layer dense model. The hidden dimensions of each token is set to 2048, and the hidden size of the FFN layer is set to 16384. We use the Group-Query Attention (GQA) [2], which contains 32 query heads and 8 key/value heads, where the hidden dimension of each head is set to 64. We train this model for 500B tokens. For the settings of our proposed *PHD* series, we pretrain two-variants of *PHD-CSWA* listed as follows:

- *PHD-CSWA-2-16-32*, where we repeat the training tokens twice. We keep a local window of 16 tokens and set the chunksize to 32 tokens.
- *PHD-CSWA-3-16-32*, where we repeat the training tokens 3 times. The local window size and chunk size are set to the same values with *PHD-CSWA-2-16-32*.

PHD-CSWA introduce consistent performance improvement across various benchmarks. We presents the training curves in Figure 5 and the main results in Table 1. Our proposed *PHD-CSWA-2-16-32* introduces an

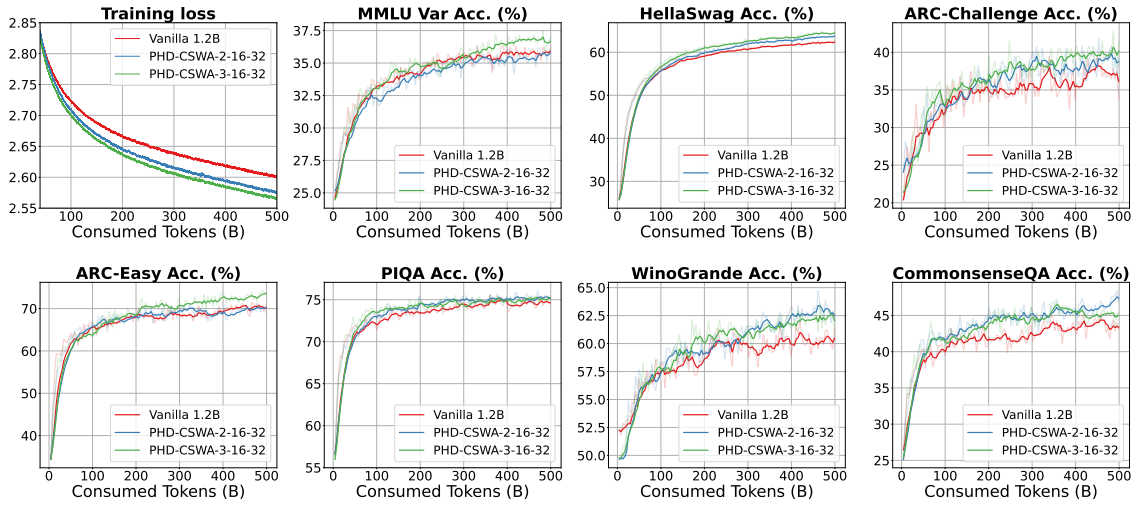


Figure 5 Training curves of *PHD-CSWA*variants and baseline model on OLMo2-1.2B. We smooth these metrics via exponential moving average with weight 0.99 for loss and 0.7 for downstream tasks.

	loss↓	MMLU-V↑	Hella.↑	ARC-C↑	ARC-E↑	PIQA↑	Wino.↑	Comm.↑	Avg.↑
Vanilla 1.2B	2.601	35.9	62.3	36.1	70.3	74.6	60.5	43.4	54.7
PHD-CSWA-2-16-32	2.576	35.8	63.7	38.8	70.2	75.2	62.5	47.4	56.2
PHD-CSWA-3-16-32	2.567	36.7	64.5	40.2	73.5	75.2	62.1	45.0	56.7

Table 1 Performance evaluation of 1.2B parameter dense models using our PHD-CSWA variants with scaling factors $K \in \{2, 3\}$. The window size W is set to 16 and the chunk size C is set to 32 for our proposed *PHD-CSWA* variants. Evaluated benchmarks include: MMLU Var (MMLU-V), Hellaswag (Hella.), ARC-Challenge (ARC-C), ARC-Easy (ARC-E), PIQA, Winogrande (Wino.), and Commonsense QA (Comm.).

average of 1.5% accuracy improvement across these benchmarks and 0.025 decrease in training loss, while *PHD-CSWA-3-16-32* introduces an average of 2.0% accuracy improvement and 0.034 decrease in training loss.

3.3 Ablation Studies

We perform comprehensive ablation studies using a 550M-sized dense transformer model. The architecture consists of 16 layers with the hidden dimension set to 1536, and the hidden size of the FFN layer set to 8192. We adopt GQA, with 16 query heads and 4 key/value heads. We train each model for 300B tokens.

3.3.1 Chunk-wise Sliding Window Attention

Since the chunk-wise sliding window attention is a compensation for accelerating the pre-filling stage, we are interested in the performance drop that CSWA may introduce. Meanwhile, we also study the performance difference when different window sizes and chunk sizes are chosen.

Training Details. We keep the scaling factor $K = 2$, and vary the window size in $\{1, 2, 4, 16\}$. For the largest window size ($W = 16$), we conduct additional experiments to study the effect of different chunk sizes, where we set the chunk size to 16, 32, and no chunk at all.

Large window size leads to better performance, but the improvement converges very fast. As shown in Figure 6, While expanding the window from $W = 1$ to $W = 4$ yields significant reductions in both training and validation loss, further increasing to $W = 16$ provides only marginal improvements. This suggests that maintaining a small window of local KV cache for *hidden decoding tokens* achieves nearly optimal performance while remaining computationally efficient and hardware-friendly.

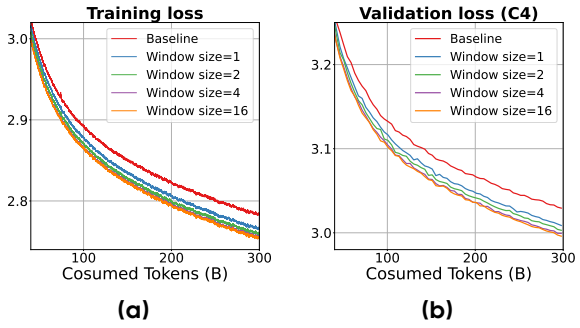


Figure 6 Ablation studies on window size.

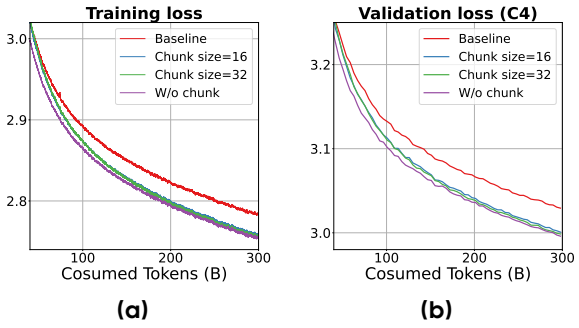


Figure 7 Ablation studies on chunk size.

Introducing chunks for reducing pre-filling overhead leads to little performance degradation. As shown in Figure 7, the introduction of chunks causes only negligible differences in both training and validation losses. We also observe a consistent trend where larger chunk sizes yield progressively better results. Based on this analysis, we select $C = 32$ as the optimal balance between computational efficiency and model performance for all subsequent experiments.

3.4 Decoding Token Scaling

In this section, we analyze the scaling performance of *PHD* and *PHD-SWA* to analyze the performance of scaling decoding computation.

Training Details We use the same 550M model configuration in this section. We set the window size W to 16 and vary the scaling factor K in $\{2, 3, 5\}$. For local window size, we set the window size to 16 for all experiments.

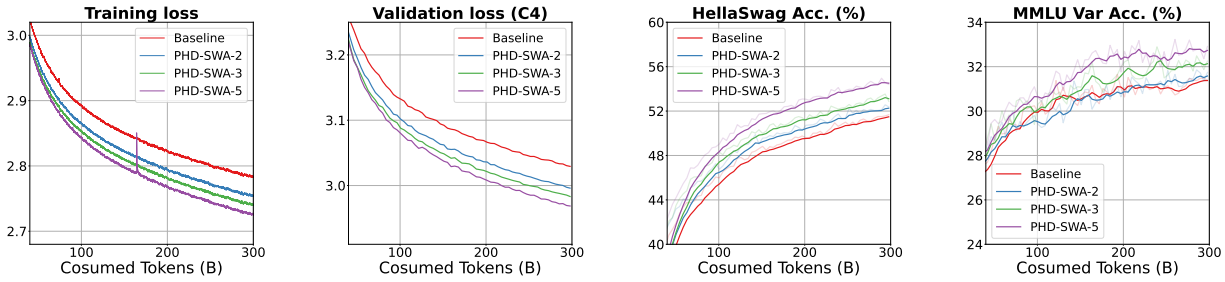


Figure 8 The scaling behavior of *PHD-SWA-K-16-*∞.

Performance of PHD-SWA scales effectively when increasing the scaling factor. As shown in Figure 8, we can see that using a fixed window size, the loss curve and downstream performance scales effectively w.r.t. the token repeating times. By setting the scaling factor to 5, we achieve near 0.06 loss decrease with notable downstream performance improvement. Quantitative results in Table 2 reveal an average accuracy improvement of 1.8% across all benchmarks when scaling to $K = 5$, confirming the effectiveness of our approach for more aggressive scaling.

3.5 Pre-filling Speed and Decoding Speed

In this section, we evaluate both the pre-filling time and decoding latency, showing that our proposed *PHD* series introduce only marginal latency overhead during the inference stage. The experiments are conducted on 550M-sized model using a single A100 GPU, and the decoding batch size is set to 1.

	Loss↓	MMLU↑	Hella.↑	ARC-C↑	ARC-E↑	PIQA↑	Wino.↑	Comm.↑	Avg.↑
Vanilla 550M	2.782	31.4	51.5	30.1	62.5	71.9	56.8	40.6	49.3
PHD-SWA-2-16-∞ 550M	2.753	31.6	52.3	31.5	64.2	71.5	56.6	42.7	50.1
PHD-SWA-3-16-∞ 550M	2.739	32.1	53.1	31.5	64.6	71.5	56.9	41.7	50.2
PHD-SWA-5-16-∞ 550M	2.725	32.7	54.5	34.7	65.0	72.3	56.2	42.6	51.1

Table 2 Decoding computation scaling trend on *PHD-SWA*. The downstream performance scales w.r.t. the increase of decoding computation.

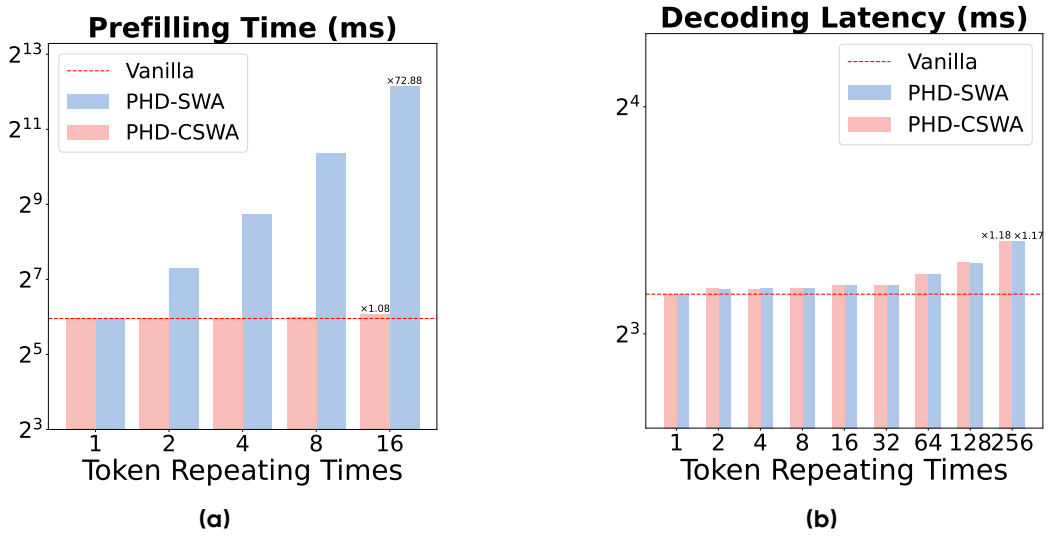


Figure 9 (a) Prefilling time and (b) decoding latency on different models when the repeating times is varied.

Our proposed PHD series introduce marginal inference overhead. As shown in Figure 9a, *PHD-SWA* introduce superlinear pre-filling overhead when the sequence length increases, while *PHD-CSWA* only incurs a small increase compared to the vanilla baseline. Figure 9b presents the decoding latency when we vary the scaling factor K in $\{1, 2, 4, 8, 16, 32, 64, 128, 256\}$. Since decoding latency is limited by memory bound, both *PHD-SWA* and *PHD-CSWA* introduces minimal overhead when K is increased. Specifically, setting K to 256 only leads to no more than 20% increase in decoding latency.

4 Related Works

The computational and memory challenges of scaling transformer models, especially in terms of attention mechanisms, have led to numerous research efforts focusing on sparse attention patterns, KV cache optimization, and efficient inference techniques. Our Parallel Hidden Decoding Transformer (PHD) builds upon and extends these approaches in novel ways. Below, we categorize and analyze relevant works in this space, drawing comparisons with our approach.

4.1 Sparse Attention Mechanisms

Sparse attention techniques aim to reduce the quadratic complexity of attention by focusing on the most informative token interactions. These approaches can be broadly categorized into three types, each with distinct relationships to our work:

Fixed Pattern Sparse Attention. These methods use predefined patterns to constrain attention computation. Notable examples include sliding window attention in Mistral [24] and Phi-3 [1], which restrict attention to local contexts; dilated attention [8, 15, 47], which attends to tokens at increasing intervals; and mixed

patterns as seen in Longformer [3] and BigBird [58], which combine local attention with global tokens. While our *PHD*shares the use of predefined attention patterns, particularly in our *PHD-SWA*variant which employs sliding window attention for *hidden decoding tokens*, we uniquely apply these patterns only to the repeated tokens, maintaining full attention for the original tokens and thus preserving model quality while gaining efficiency.

Data-Dependent Sparse Attention. Unlike fixed patterns, these approaches dynamically determine attention patterns based on input characteristics. Quest [50] proposes data-dependent block-wise sparse attention that adaptively selects blocks based on token-block similarity. Other examples include SpAtten [53] and SparQ [42], which leverage the dynamic nature of attention to predict sparse patterns. While these methods offer adaptive flexibility, they introduce substantial overhead in estimating the sparse patterns, limiting their effectiveness for long-context scenarios. In contrast, our PHD approach uses static patterns that require no runtime estimation overhead, making it more suitable for practical deployment while still achieving performance improvements through increased computational depth.

Training-Native Sparse Attention. To address the inconsistency between training and inference in post-hoc sparse attention, Native Sparse Attention (NSA) [57] incorporates block sparse patterns during the training stage itself. This approach achieves improvements in both downstream performance and efficiency. Similar to NSA, our PHD method integrates efficient attention patterns during training, but unlike NSA's focus on general block sparsity, we specifically target *hidden decoding tokens* with tailored attention patterns, allowing us to maintain the same KV cache footprint as the original transformer while significantly improving performance.

4.2 KV Cache Optimization

As models scale to handle longer contexts, KV cache management becomes increasingly critical for efficient inference:

KV Cache Reduction. Various approaches attempt to reduce KV cache size, including H2O [61], which identifies and discards less important KV cache entries; StreamingLLM [56], which employs sink attention for handling streaming inputs; SnapKV [32], which merges similar tokens; and compression-based methods like LongLLMLingua [26, 40]. Unlike these methods which primarily focus on compressing existing KV caches post-hoc, our PHD approach fundamentally rethinks the relationship between computation and KV cache by sharing KV cache across repeated tokens, maintaining the same KV cache size as the original transformer while increasing computational depth for improved performance.

KV Cache Management. Beyond reduction, effective management of the KV cache is crucial. PagedAttention [30] optimizes KV cache allocation and access patterns, while FlashDecoding [14] and FlashDecoding++ [23] enhance the efficiency of attention computations during decoding. Our *PHD*-Transformer complements these methods by focusing on the efficient utilization of the KV cache through controlled sharing patterns, and could potentially be combined with these management techniques for further efficiency gains.

Attention Kernels and Hardware Optimization. Specialized attention implementations like FlashAttention series [12, 13, 45] and RingAttention [5, 35] optimize memory access patterns to accelerate attention computation. Our *PHD*approach is orthogonal to these kernel optimizations and could leverage them for implementation, potentially providing compounded efficiency gains while addressing the core issue of scaling computational depth without proportional KV cache increases.

4.3 Latent Thinking Transformers

Hidden Decoding Tokens Insertion. With the success of Chain-of-thought reasoning [55] in the inference stage, researchers have long been interested in developing pretraining language models that can reason themselves. In the early stage of this topic, Goyal et al. [17] propose to insert learnable *pause* tokens randomly in the pretraining sequence and observes improvement on reasoning benchmarks including GSM8K [11],

NaturalQ [29], CommonsenseQA [49], etc. Quiet-Star [59] further proposes a reinforce framework to insert thinking tokens via online exploration. More recently, researchers [20, 36, 48] have converted these discrete thinking tokens to continuous signals and are proven effective beyond reasoning benchmarks. However, their applicability is largely limited due to the linear increase of KV cache size. Our proposed *PHD*also leverages continuous signals, but introduces well-designed KV cache management to maintain an acceptable increase of KV cache, leading to applicability during inference.

Recurrent Latent Thinking Layers. Another line of researches seek for recurrent approaches in reusing model parameters. Both Chen et al. [7], Geiping et al. [16] propose to use recurrent transformer block or transformer layer to scale the computation of decoding tokens, either statically or adaptively. However, recurrent models cannot be used in parallel generation, leading to limited efficiency during inference.

In summary, our *PHD*-Transformer series introduce a unique approach to model scaling by focusing on computational depth without proportional increases in KV cache size. By combining token repetition with efficient attention patterns specifically for *hidden decoding tokens*, PHD achieves performance scaling without the prohibitive memory costs typically associated with length scaling approaches. Leveraging insights from attention pattern analysis in works like MInference [25], we develop a practical pretraining length scaling technique that addresses a key bottleneck in current LLM inference where models are typically memory-bound rather than compute-bound.

5 Conclusion

In this paper, we establish pre-training length scaling as an efficient paradigm for enhancing transformer models through our Parallel Hidden Decoding (PHD) framework. By strategically repeating input tokens while retaining only original tokens in the KV cache, *PHD*-Transformer achieves significant performance gains without increasing the KV cache size. The *PHD-SWA* variant further preserves local dependencies through sliding window attention, while *PHD-CSWA* eliminates linear pre-filling latency growth through chunk-wise sliding window attention. Experiments demonstrate consistent improvements across multiple benchmarks, validating that length scaling can be both effective and resource-efficient during inference.

References

[1] Marah I Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat S. Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, Caio César Teodoro Mendes, Weizhu Chen, Vishrav Chaudhary, Parul Chopra, Allie Del Giorno, Gustavo de Rosa, Matthew Dixon, Ronen Eldan, Dan Iter, Amit Garg, Abhishek Goswami, Suriya Gunasekar, Emman Haider, Junheng Hao, Russell J. Hewett, Jamie Huynh, Mojan Javaheripi, Xin Jin, Piero Kauffmann, Nikos Karampatziakis, Dongwoo Kim, Mahoud Khademi, Lev Kurilenko, James R. Lee, Yin Tat Lee, Yuanzhi Li, Chen Liang, Weishung Liu, Eric Lin, Zeqi Lin, Piyush Madan, Arindam Mitra, Hardik Modi, Anh Nguyen, Brandon Norick, Barun Patra, Daniel Perez-Becker, Thomas Portet, Reid Pryzant, Heyang Qin, Marko Radmilac, Corby Rosset, Sambudha Roy, Olatunji Ruwase, Olli Saarikivi, Amin Saied, Adil Salim, Michael Santacroce, Shital Shah, Ning Shang, Hiteshi Sharma, Xia Song, Masahiro Tanaka, Xin Wang, Rachel Ward, Guanhua Wang, Philipp Witte, Michael Wyatt, Can Xu, Jiahang Xu, Sonali Yadav, Fan Yang, Ziyi Yang, Donghan Yu, Chengruidong Zhang, Cyril Zhang, Jianwen Zhang, Li Lyna Zhang, Yi Zhang, Yue Zhang, Yunan Zhang, and Xiren Zhou. Phi-3 technical report: A highly capable language model locally on your phone. CoRR, abs/2404.14219, 2024. doi: 10.48550/ARXIV.2404.14219. URL https://doi.org/10.48550/arXiv.2404.14219.

[2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 4895–4901, 2023.

[3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. CoRR, abs/2004.05150, 2020. URL https://arxiv.org/abs/2004.05150.

[4] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In Proceedings of the AAAI conference on artificial intelligence, volume 34, pages 7432–7439, 2020.

[5] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. Striped attention: Faster ring attention for causal transformers. CoRR, abs/2311.09431, 2023. doi: 10.48550/ARXIV.2311.09431. URL https://doi.org/10.48550/arXiv.2311.09431.

[6] Guoxin Chen, Minpeng Liao, Chengxi Li, and Kai Fan. Step-level value preference optimization for mathematical reasoning. arXiv preprint arXiv:2406.10858, 2024.

[7] Yilong Chen, Junyuan Shang, Zhenyu Zhang, Yanxi Xie, Jiawei Sheng, Tingwen Liu, Shuohuan Wang, Yu Sun, Hua Wu, and Haifeng Wang. Inner thinking transformer: Leveraging dynamic depth scaling to foster adaptive internal thinking. arXiv preprint arXiv:2502.13842, 2025.

[8] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. CoRR, abs/1904.10509, 2019. URL http://arxiv.org/abs/1904.10509.

[9] Aidan Clark, Diego de Las Casas, Aurelia Guy, Arthur Mensch, Michela Paganini, Jordan Hoffmann, Bogdan Damoc, Blake Hechtman, Trevor Cai, Sebastian Borgeaud, et al. Unified scaling laws for routed language models. In International conference on machine learning, pages 4057–4086. PMLR, 2022.

[10] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. arXiv preprint arXiv:1803.05457, 2018.

[11] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[12] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=mZn2Xyh9Ec.

[13] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

[14] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference, October 2023. URL https://crfm.stanford.edu/2023/10/12/flashdecoding.html. Accessed: 2024-9-29.

[15] Jiayu Ding, Shuming Ma, Li Dong, Xingxing Zhang, Shaohan Huang, Wenhui Wang, Nanning Zheng, and Furu Wei. Longnet: Scaling transformers to 1, 000, 000, 000 tokens. CoRR, abs/2307.02486, 2023. doi: 10.48550/ARXIV.2307.02486. URL https://doi.org/10.48550/arXiv.2307.02486.

[16] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. arXiv preprint arXiv:2502.05171, 2025.

[17] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. In The Twelfth International Conference on Learning Representations.

[18] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948, 2025.

[19] Ji-Eun Han, Jun-Seok Koh, Hyeon-Tae Seo, Du-Seong Chang, and Kyung-Ah Sohn. Psydial: personality-based synthetic dialogue generation using large language models. arXiv preprint arXiv:2404.00930, 2024.

[20] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. arXiv preprint arXiv:2412.06769, 2024.

[21] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.

[22] Danny Hernandez, Jared Kaplan, Tom Henighan, and Sam McCandlish. Scaling laws for transfer. arXiv preprint arXiv:2102.01293, 2021.

[23] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhan Dong, and Yu Wang. Flashdecoding++: Faster large language model inference on gpus. CoRR, abs/2311.01282, 2023. doi: 10.48550/ARXIV.2311.01282. URL https://doi.org/10.48550/arXiv.2311.01282.

[24] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. CoRR, abs/2310.06825, 2023. doi: 10.48550/ARXIV.2310.06825. URL https://doi.org/10.48550/arXiv.2310.06825.

[25] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir Abdi, Dongsheng Li, Chin-Yew Lin, et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. Advances in Neural Information Processing Systems, 37:52481–52515, 2024.

[26] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024, pages 1658–1677. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.91. URL https://doi.org/10.18653/v1/2024.acl-long.91.

[27] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? arXiv preprint arXiv:2310.06770, 2023.

[28] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361, 2020.

[29] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. Transactions of the Association for Computational Linguistics, 7:453–466, 2019.

[30] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace, editors, Proceedings

of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

[31] Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, Cheng Zhu, Chunhao Zhang, Congchao Guo, Da Chen, Dong Li, et al. Minimax-01: Scaling foundation models with lightning attention. arXiv preprint arXiv:2501.08313, 2025.

[32] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: LLM knows what you are looking for before generation. CoRR, abs/2404.14469, 2024. doi: 10.48550/ARXIV.2404.14469. URL https://doi.org/10.48550/arXiv.2404.14469.

[33] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434, 2024.

[34] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.

[35] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. CoRR, abs/2310.01889, 2023. doi: 10.48550/ARXIV.2310.01889. URL https://doi.org/10.48550/arXiv.2310.01889.

[36] Amirkeivan Mohtashami, Matteo Pagliardini, and Martin Jaggi. Cotformer: More tokens with attention make up for less depth. In Workshop on Advancing Neural Network Training: Computational Efficiency, Scalability, and Resource Optimization (WANT@ NeurIPS 2023), 2023.

[37] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, et al. 2 olmo 2 furious. arXiv preprint arXiv:2501.00656, 2024.

[38] OpenAI. Learning to reason with llms, 2024. URL https://openai.com/index/learning-to-reason-with-llms/.

[39] OpenAI. Learning to reason with llms, 2025. URL https://openai.com/index/openai-o3-mini/.

[40] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024, pages 963–981. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.FINDINGS-ACL.57. URL https://doi.org/10.18653/v1/2024.findings-acl.57.

[41] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In First Conference on Language Modeling, 2024.

[42] Luka Ribar, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. Sparq attention: Bandwidth-efficient llm inference. arXiv preprint arXiv:2312.04985, 2023.

[43] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. Communications of the ACM, 64(9):99–106, 2021.

[44] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.

[45] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. CoRR, abs/2407.08608, 2024. doi: 10.48550/ARXIV.2407.08608. URL https://doi.org/10.48550/arXiv.2407.08608.

[46] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

[47] Han Shi, Jiahui Gao, Xiaozhe Ren, Hang Xu, Xiaodan Liang, Zhenguo Li, and James Tin-Yau Kwok. Sparsebert: Rethinking the importance analysis in self-attention. In Marina Meila and Tong Zhang, editors, Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event, volume

139 of Proceedings of Machine Learning Research, pages 9547–9557. PMLR, 2021. URL http://proceedings.mlr.press/v139/shi21a.html.

[48] Jihoon Tack, Jack Lanchantin, Jane Yu, Andrew Cohen, Ilia Kulikov, Janice Lan, Shibo Hao, Yuandong Tian, Jason Weston, and Xian Li. Llm pretraining with continuous concepts. arXiv preprint arXiv:2502.08524, 2025.

[49] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. Commonsenseqa: A question answering challenge targeting commonsense knowledge. arXiv preprint arXiv:1811.00937, 2018.

[50] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. In International Conference on Machine Learning, pages 47901–47911. PMLR, 2024.

[51] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. arXiv preprint arXiv:2312.11805, 2023.

[52] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530, 2024.

[53] Hanrui Wang, Zhekai Zhang, and Song Han. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In IEEE International Symposium on High-Performance Computer Architecture, HPCA 2021, Seoul, South Korea, February 27 - March 3, 2021, pages 97–110. IEEE, 2021. doi: 10.1109/HPCA51647.2021.00018. URL https://doi.org/10.1109/HPCA51647.2021.00018.

[54] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In The Thirteenth International Conference on Learning Representations, 2024.

[55] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. Advances in neural information processing systems, 35:24824–24837, 2022.

[56] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=NG7sS51zVF.

[57] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, YX Wei, Lean Wang, Zhiping Xiao, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. arXiv preprint arXiv:2502.11089, 2025.

[58] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/c8512d142a2d849725f31a9a7a361ab9-Abstract.html.

[59] Eric Zelikman, Georges Raif Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah Goodman. Quiet-star: Language models can teach themselves to think before speaking. In First Conference on Language Modeling, 2024.

[60] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? arXiv preprint arXiv:1905.07830, 2019.

[61] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. Advances in Neural Information Processing Systems, 36:34661–34710, 2023.